Universidad Nacional Autónoma de México

Licenciatura en Informática

Desarrollo de Aplicaciones Móviles

Cristian Cardoso Arellano

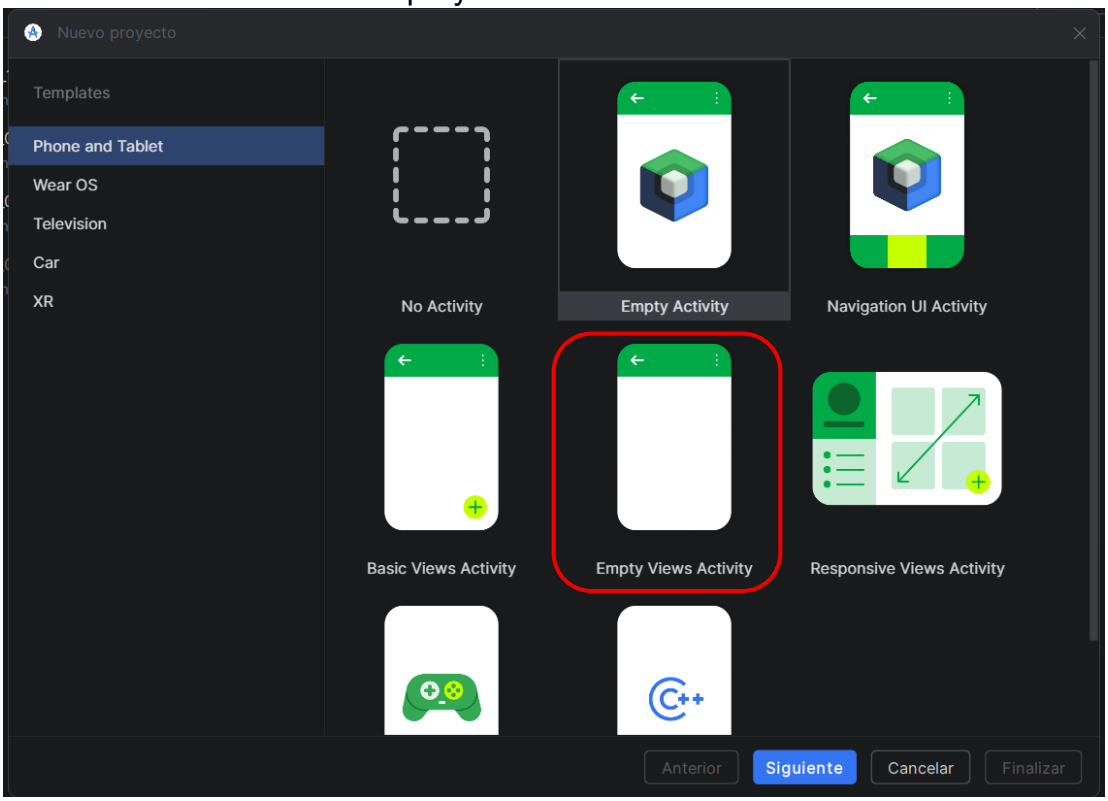
Miguel Alejandro Ramírez Meza

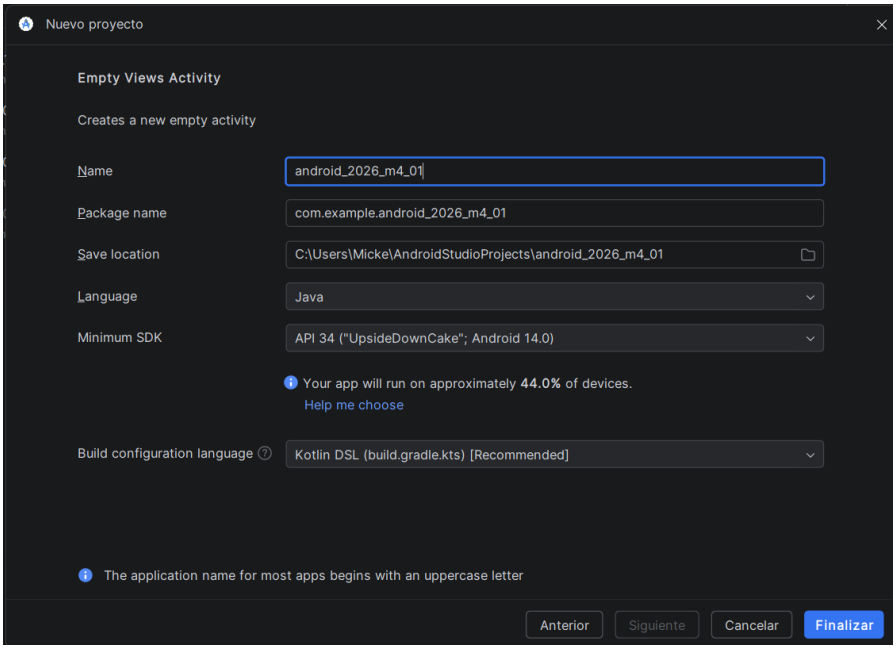
Unidad 4 Actividad 1

Fecha: 14 de marzo de 2026

1. **1. Crea una activity simple en Android Studio en lenguaje de programación Java.**
2. **Imprimir un mensaje personalizado en cada uno de los métodos del ciclo de vida de una actividad.**
   - **onCreate()**
   - **onStart()**
   - **onPause()**
   - **onRestart()**
   - **onResume()**
   - **onStop()**
   - **onDestroy()**
3. **Subir un sólo archivo en formato PDF en el repositorio de github creado por el alumno.**
   - **android_2026_m3_01.pdf**
   - **código fuente.**

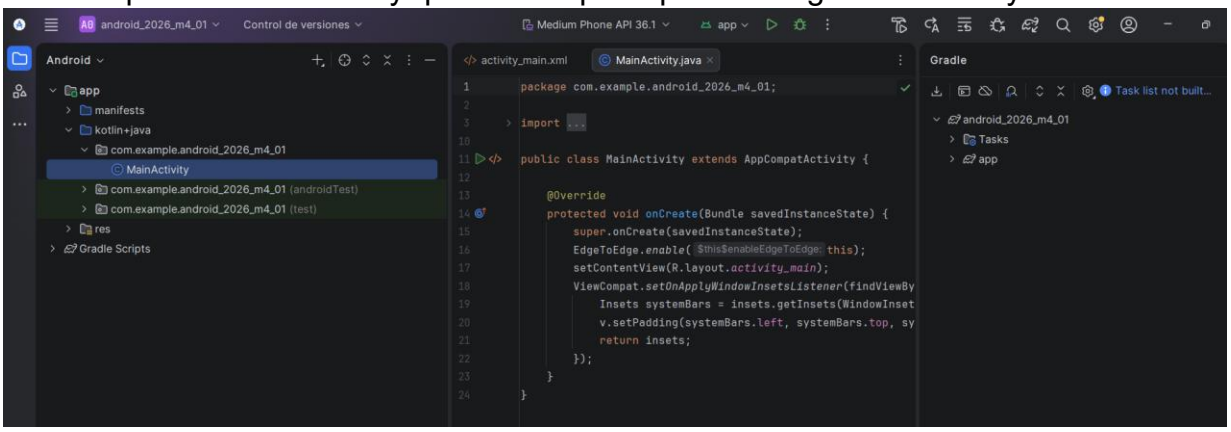
Primero iniciamos un nuevo proyecto en Android Studio



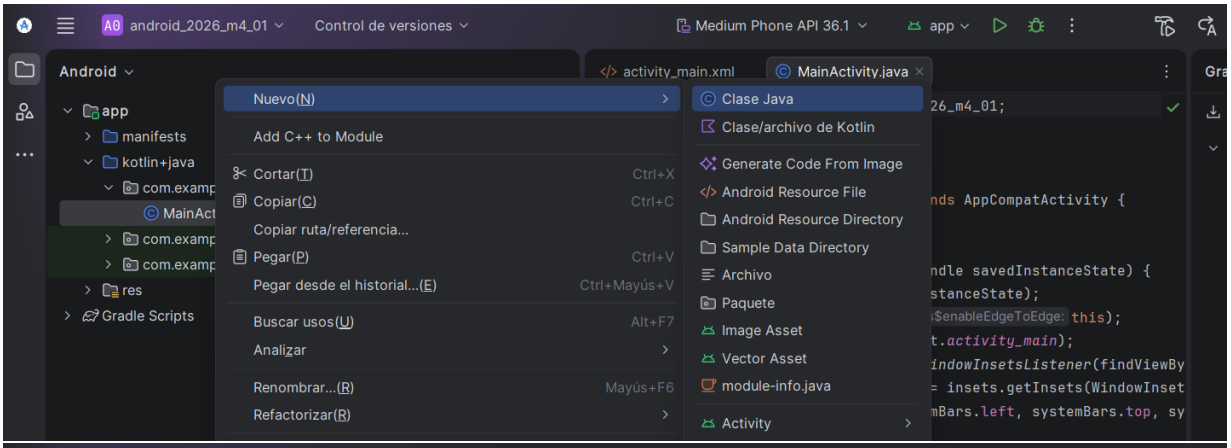


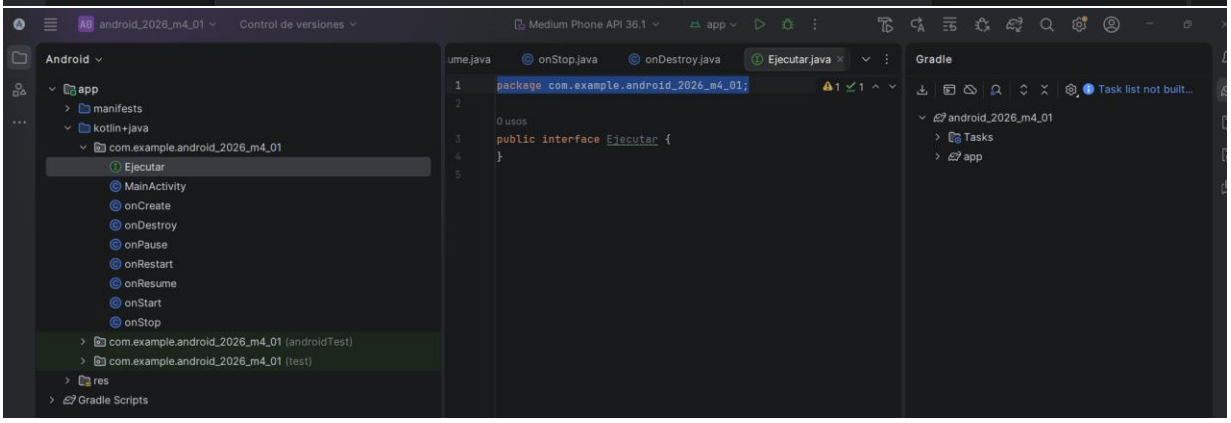
Para integrar los archivos descargados de la calculadora en el proyecto debemos desplegar la carpeta kotlin + java

Posteriormente, seleccionamos la carpeta con el nombre del proyecto que no tiene etiquetas adicionales y que se compone por el código MainActivity

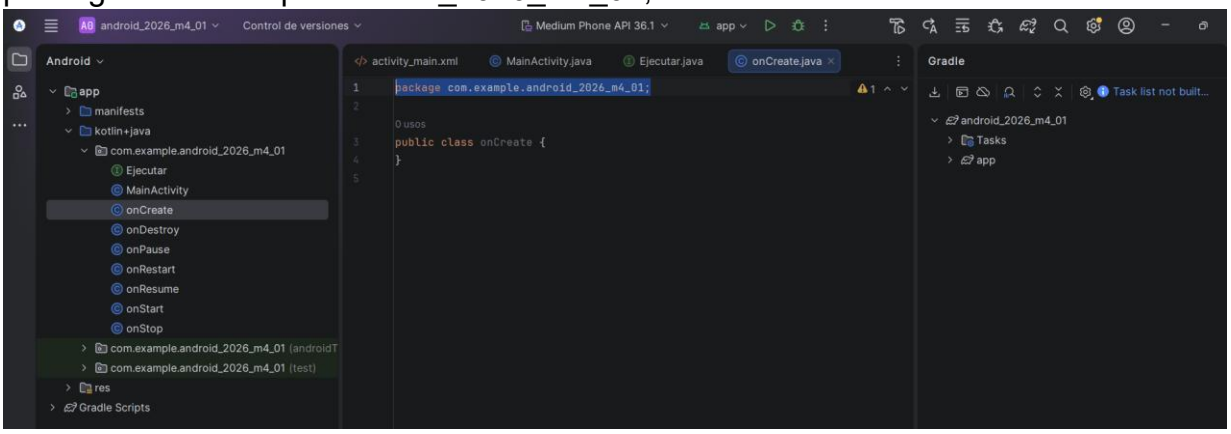


Aquí, debemos de dar clic derecho sobre la carpeta y seleccionar Nuevo > Clase Java y crear 8 archivos, de los cuales 7 perteneces a las Clases de las funciones y una adicional para la interfaz donde se invocan las clases





A continuación, debemos de verificar que al abrir cada archivo en Android Studio, la primera liga del código diga:
package com.example.android_2026_m4_01;



- Definir el código para cada archivo

Ahora ya podemos iniciar a desarrollar el código:

  - MainActivity

package com.example.android_2026_m4_01;

import android.os.Bundle;
import android.text.InputType;

```
import android.widget.Button;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private EditText inputOption;
    private Button btnEjecutar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        LinearLayout layout = new LinearLayout(this);
        layout.setOrientation(LinearLayout.VERTICAL);
        layout.setPadding(50,150,50,50);

        inputOption = new EditText(this);
        inputOption.setHint("Ingrese opción (1-7)");
        inputOption.setInputType(InputType.TYPE_CLASS_NUMBER);

        btnEjecutar = new Button(this);
        btnEjecutar.setText("Ejecutar");

        layout.addView(inputOption);
        layout.addView(btnEjecutar);

        setContentView(layout);

        btnEjecutar.setOnClickListener(v -> {

            String opcion = inputOption.getText().toString();
            ejecutarMenu(opcion);

        });
    }

    private void ejecutarMenu(String opcion){

        String opcionLimpia = opcion.trim();
```

```
        if(opcionLimpia.isEmpty()){
            Toast.makeText(this,"Ingrese una
opción",Toast.LENGTH_SHORT).show();
            return;
        }

        int numero;

        try{
            numero = Integer.parseInt(opcionLimpia);
        }catch(Exception e){
            Toast.makeText(this,"Debe ingresar un
número",Toast.LENGTH_SHORT).show();
            return;
        }

        Ejecutar accion = null;

        switch(numero){

            case 1:
                accion = new onCreateAction();
                break;

            case 2:
                accion = new onStart();
                break;

            case 3:
                accion = new onResume();
                break;

            case 4:
                accion = new onPause();
                break;

            case 5:
                accion = new onStop();
                break;

            case 6:
                accion = new onRestart();
                break;
```

```
            case 7:
                accion = new onDestroy();
                break;

            default:
                Toast.makeText(this,"Opción
inválida",Toast.LENGTH_SHORT).show();
                return;
        }

        if(accion != null){
            accion.ejecutar();
            inputOption.setText("");
        }
    }
}
```

- Ejecutar

```
package com.example.android_2026_m4_01;
import android.util.Log;

public interface Ejecutar {
    void ejecutar();
}
```

- onCreateAction

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onCreateAction implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("createAction", "Metodo onCreateAction ejecutado correctamente");
    }
}
```

- onDestroy

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onDestroy implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("onDestroy", "Metodo onDestroy ejecutado correctamente");
```

```
    }
}
```

- onPause

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onPause implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("pausa", "Metodo onPause ejecutado correctamente");
    }
}
```

- onRestart

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onRestart implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("reiniciar", "Metodo onRestart ejecutado correctamente");
    }
}
```

- onResume

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onResume implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("resumen", "Metodo onResume ejecutado correctamente");
    }
}
```

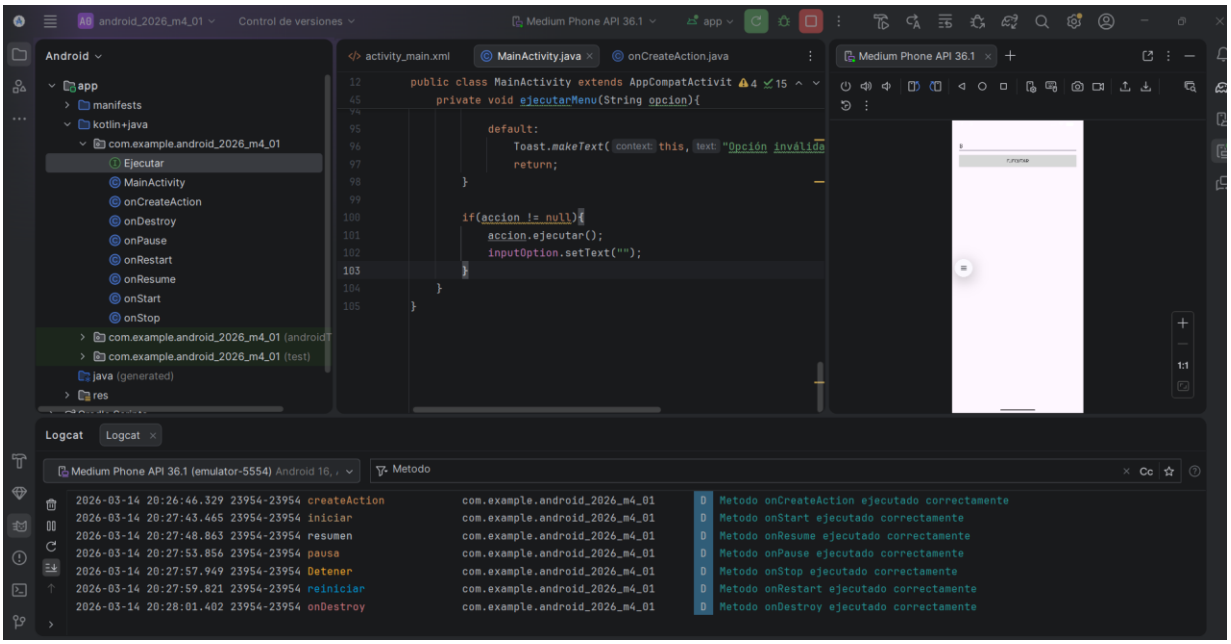
- onStart

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onStart implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("iniciar", "Metodo onStart ejecutado correctamente");
```

```
    }
}
```

- onStop

```
package com.example.android_2026_m4_01;
import android.util.Log;

public class onStop implements Ejecutar {
    @Override
    public void ejecutar() {
        Log.d("Detener", "Metodo onStop ejecutado correctamente");
    }
}
```



Conclusión

Para hacer esta actividad primero pensé en realizar todo el código en un solo archivo, mediante programación estructurada. Sin embargo, en la actualidad no es realista; ya que se necesita que las aplicaciones sean escalables. Por ello, tuve que utilizar un enfoque de Programación Orientada a Objetos y definir varias clases.

Esta actividad se me complicó mucho, ya que el ingresar las opciones y apretar Enter, Android Studio no reconoce el método OnEditorActionListener. Por esto tuve que modificar el código de mi proyecto en varias ocasiones, pero el error continuaba.

Para encontrar la solución a este problema, tuve que ayudarme de chatgpt, ya que Gemini tiene limitantes en cuanto a desarrollar código. Fue mediante la IA, que me recomendó implementar un botón para evitar el uso de la tecla. Este sencillo cambio garantizó la compatibilidad con todos los dispositivos que vayan a ejecutarlo

Fuente consultada:

- “ChatGPT”, https://chatgpt.com/s/t_69b63a13ce808191b15879d9eab8b0e5